ONLINE PARKING MANAGEMENT SYSTEM

AI23431- WEBTECHNOLOGY AND MOBILE APPLICATION

PROJECT REPORT

SUBMITTED BY

SHIVANESH S (241501200)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS),

CHENNAI-602 105

APRIL 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this Project Thesis titled "**ONLINE PARKING MANAGEMENT SYSTEM**" is the Bonafide work of SHIVANESH S (2116241501200) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported here in does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Mrs . DURGA DEVI B

Assistant Professor,

Department of Artificial Intelligence and Machine Learning,

Rajalakshmi Engineering College,

Chennai - 602105.

Submitted to Project Viva-Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, **Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. K. SEKAR, Ph.D.**, Professor and Head of the Department of Artificial Intelligence and Machine Learning for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mrs. DURGA DEVI B,** Department of Artificial Intelligence and Machine Learning for her useful tips during our review to build our project.

Shivanesh S - 241501200

ABSTRACT

The Online Parking Management System is a web-based application designed to digitize and streamline the process of managing parking slots in urban areas, commercial complexes, and institutions. With the rapid increase in vehicle ownership, finding available parking spaces has become a significant challenge. Traditional parking management methods rely on manual supervision, paper records, or first-come-first-serve systems, which often lead to inefficient space utilization and user dissatisfaction.

The proposed system allows users to register, log in securely, view available parking slots in real-time, and book slots for specific dates and durations. Each booking generates a digital record stored in a centralized database. Administrators have access to a dedicated dashboard where they can view all active bookings across all users, monitor slot availability, cancel or delete bookings, and generate operational insights.

The system is built using Node.js for backend processing, MongoDB for database management, Express.js as the web framework, and EJS along with HTML/CSS for the frontend interface. Role-based authentication ensures secure access for both regular users and administrators.

By implementing this system, parking space utilization improves, manual workload reduces, and users gain transparency and convenience in booking parking slots. The system is scalable, cost-effective, and suitable for deployment in malls, colleges, hospitals, and corporate offices.

CHAPTER 1: INTRODUCTION

1.1 General

Parking management is a critical aspect of urban infrastructure, especially in densely populated cities where vehicle density continues to rise. Inefficient parking systems lead to traffic congestion, wasted time, fuel wastage, and environmental pollution. Many parking facilities still operate manually, relying on physical tokens, paper slips, or verbal communication, which are prone to errors and inefficiencies.

The Online Parking Management System is developed to address these challenges by providing a digital platform where users can book parking slots in advance and administrators can manage parking operations efficiently.

1.2 Need for Online Parking Management System

- Real-time availability: Users can check available slots before visiting the parking location.
- Advance booking: Users can reserve slots for specific dates and durations.
- Reduced manual effort: Automates booking records, cancellations, and availability tracking.
- Centralized data: All booking information is stored in a database for easy retrieval and analysis.
- Improved user experience: Transparent, convenient, and time-saving.

1.3 Scope of the System

The system provides separate interfaces for users and administrators. Users can register, log in, book slots, view their booking history, and cancel bookings. Administrators can view all active bookings from all users, cancel or delete any booking, and monitor overall parking statistics.

The system can be deployed in:

- Shopping malls and multiplexes
- Corporate offices and IT parks
- Educational institutions
- Hospitals and healthcare facilities
- Airport and railway station parking

1.4 Organization of the Report

- Chapter 1: Introduction, need, scope
- Chapter 2: Literature Survey
- Chapter 3: Problem Formulation and Objectives
- Chapter 4: System Design (Architecture, Database, Requirements)
- Chapter 5: System Implementation (Modules, Code Snippets)
- Chapter 6: Results and Discussion (Screenshots)
- Chapter 7: Conclusion and Future Work

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

Several parking management systems have been developed over the years, ranging from simple token-based systems to advanced IoT-enabled smart parking solutions. Research in this domain focuses on optimizing space utilization, reducing search time, and improving user convenience.

2.2 Related Work

Several studies have proposed different approaches to parking management:

Sensor-based systems use ultrasonic or IR sensors to detect vehicle presence. While accurate, they require hardware installation and maintenance.

Mobile applications provide user-friendly interfaces but require users to download and install apps.

Web-based systems offer cross-platform accessibility without installation, making them suitable for institutions and commercial complexes.

The proposed system adopts a web-based approach with role-based authentication, real-time slot tracking, and centralized database management, balancing functionality with simplicity.

2.3 Summary

From the literature survey, it is evident that automated parking systems significantly improve efficiency compared to manual methods. However, advanced systems require expensive infrastructure. A web-based system offers a cost-effective, accessible, and scalable solution for parking management.

CHAPTER 3: PROBLEM FORMULATION AND OBJECTIVES

3.1 Problem Statement

In many parking facilities, users face difficulties finding available parking spaces due to lack of real-time information. Manual systems do not allow advance booking, leading to uncertainty and wasted time. Parking administrators struggle to track occupancy, manage records, and handle disputes.

There is a need for an automated system that provides:

- Real-time slot availability
- Advance booking capability
- Centralized record management
- Administrative oversight

3.2 Objectives of the System

To develop a web-based platform for online parking slot booking.

- To provide secure login and registration for users and administrators.
- To display real-time availability of parking slots.
- To allow users to book slots for specific dates and durations.
- To enable administrators to view all active bookings from all users.
- To allow cancellation and deletion of bookings by administrators.

3.3 Scope of the System

List all possible deployment scenarios with explanations:

Shopping malls

1. Corporate offices
2. Educational institutions
3. Hospitals
4. Airport/Railway stations
5. Residential complexes
6. Event parking

3.4 Comparative Analysis

Compare existing systems on parameters:

1. Cost
2. Scalability
3. User-friendliness
4. Real-time updates
5. Advance booking
6. Admin controls

3.5 Research Gaps Identified

1. Lack of role-based authentication in many systems
2. No centralized admin dashboard
3. Limited advance booking features

3.7 The Problem Domain

Parking management is a critical yet often overlooked aspect of urban infrastructure. With the rapid increase in vehicle ownership worldwide, the demand for efficient parking solutions has never been greater. According to recent studies, drivers in metropolitan cities spend an average of 20-30 minutes searching for available parking spaces, contributing to traffic congestion, fuel wastage, and environmental pollution.

In India, the situation is particularly challenging. Cities like Delhi, Mumbai, Bengaluru, and Chennai face severe parking shortages. The vehicle-to-parking space ratio in major Indian cities is estimated to be 10:1, meaning there is only one parking spot for every ten vehicles. This imbalance creates significant challenges for both vehicle owners and parking facility managers.

Real-World Statistics:

- 30% of urban traffic congestion is caused by drivers searching for parking
- Average driver spends 17 hours per year looking for parking
- $345 billion annual economic loss globally due to parking-related issues
- 40% reduction in parking space utilization in manual systems

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

System design defines the overall architecture, components, and data flow of the application. The Online Parking Management System follows a three-layer architecture: Presentation Layer (frontend), Application Layer (backend), and Database Layer (MongoDB).

4.2 System Architecture

text

[User Browser] ←→ [HTML/CSS/EJS Frontend] ←→ [Node.js + Express Backend] ←→ [MongoDB Database]

↑ ↑

[Admin Dashboard] [Authentication Middleware]

Presentation Layer: Built using HTML, CSS, JavaScript, and EJS templates. Provides separate interfaces for users and administrators.

Application Layer: Node.js with Express framework handles routing, authentication, booking logic, and database operations.

Database Layer: MongoDB stores users, bookings, and slot information.

4.3 Database Design

Collections:

users

_id, username, email, password, role, createdAt

bookings

_id, userId, slotNumber, vehicleNumber, bookingDate, duration, status, createdAt

slots

_id, slotNumber, isAvailable, pricePerHour

4.4 System Requirements

4.4.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or higher
RAM	4 GB
Storage	500 GB HDD / SSD
Input	Keyboard and Mouse
Network	Internet connection

4.4.2 Software Requirements

Component	Technology
Operating System	Windows 10 / 11
Backend	Node.js
Database	MongoDB
Frontend	HTML, CSS, JavaScript, EJS
IDE	Visual Studio Code
Browser	Google Chrome / Firefox

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 System Methodology

The system is implemented using the MVC (Model-View-Controller) architecture:

Models: Define database schemas (User, Booking, Slot)

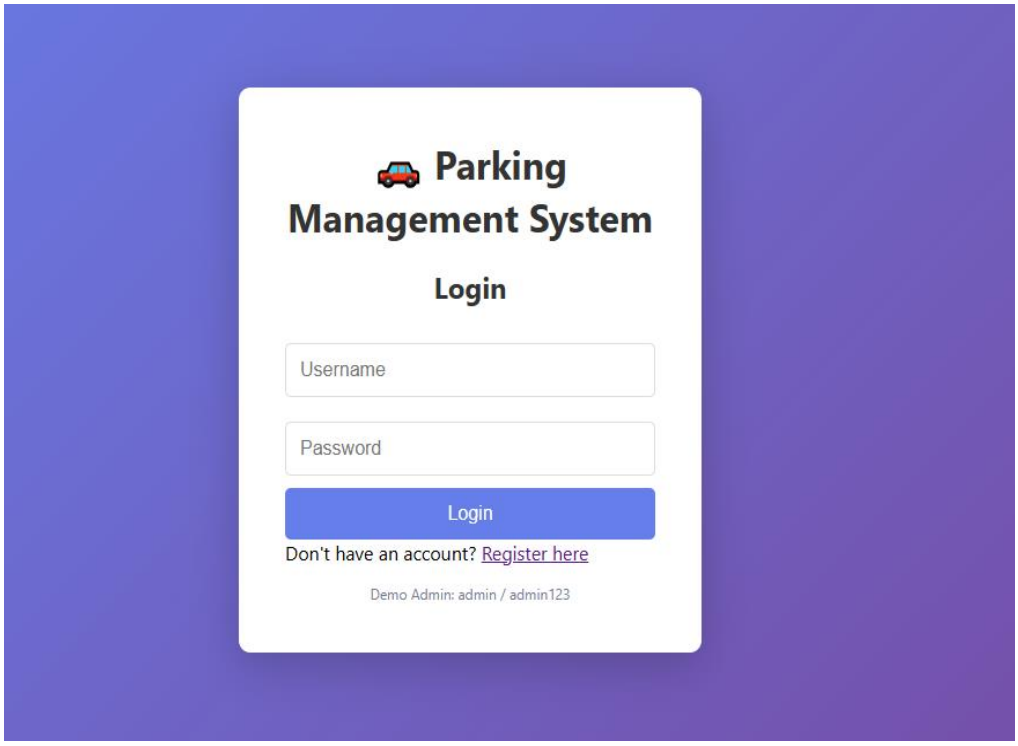
Views: EJS templates for UI pages

Controllers: Route handlers for business logic

5.2 Module Descriptions

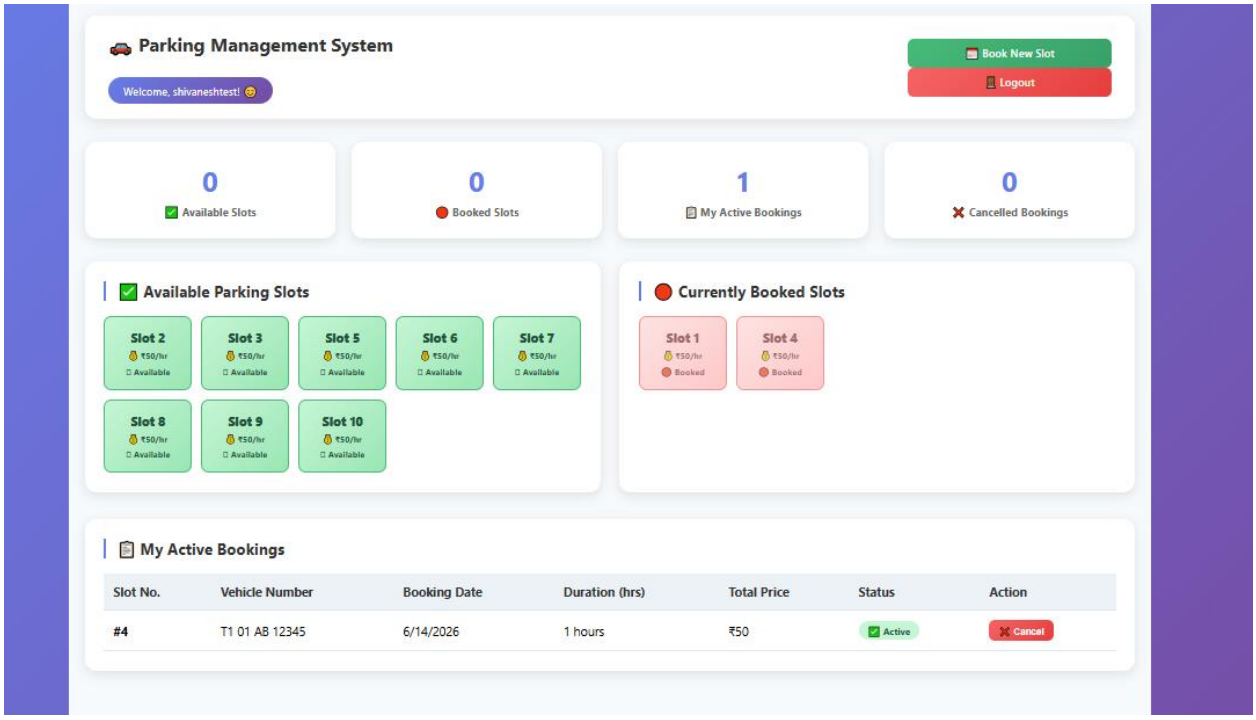
5.2.1 User Authentication Module

Users can register with username, email, and password. Passwords are hashed using bcrypt before storage. Login validates credentials and creates a session.



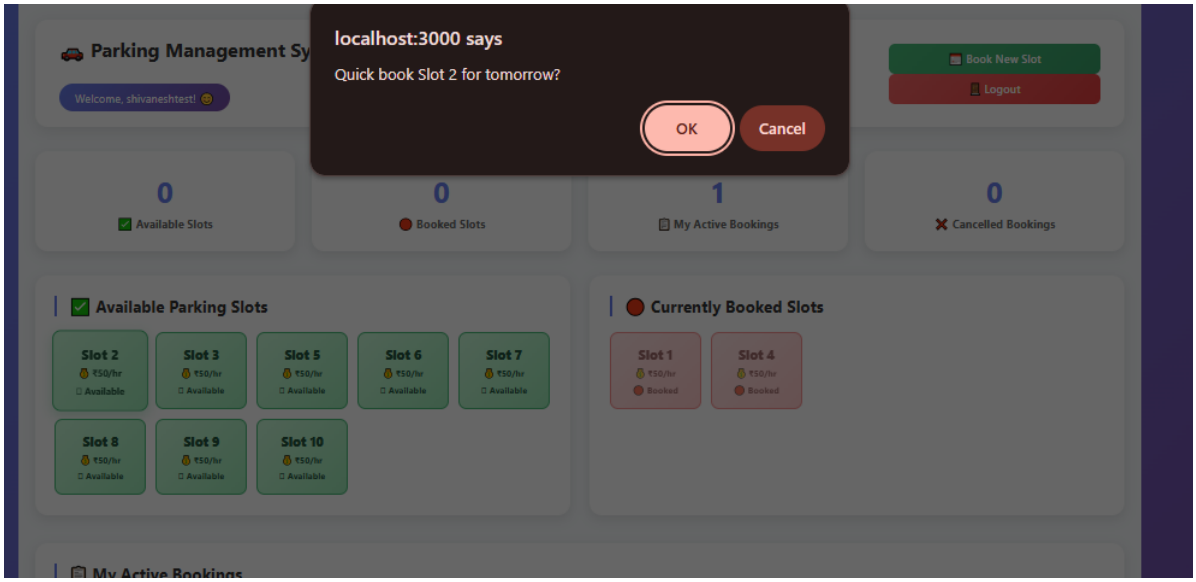
5.2.2 User Dashboard Module

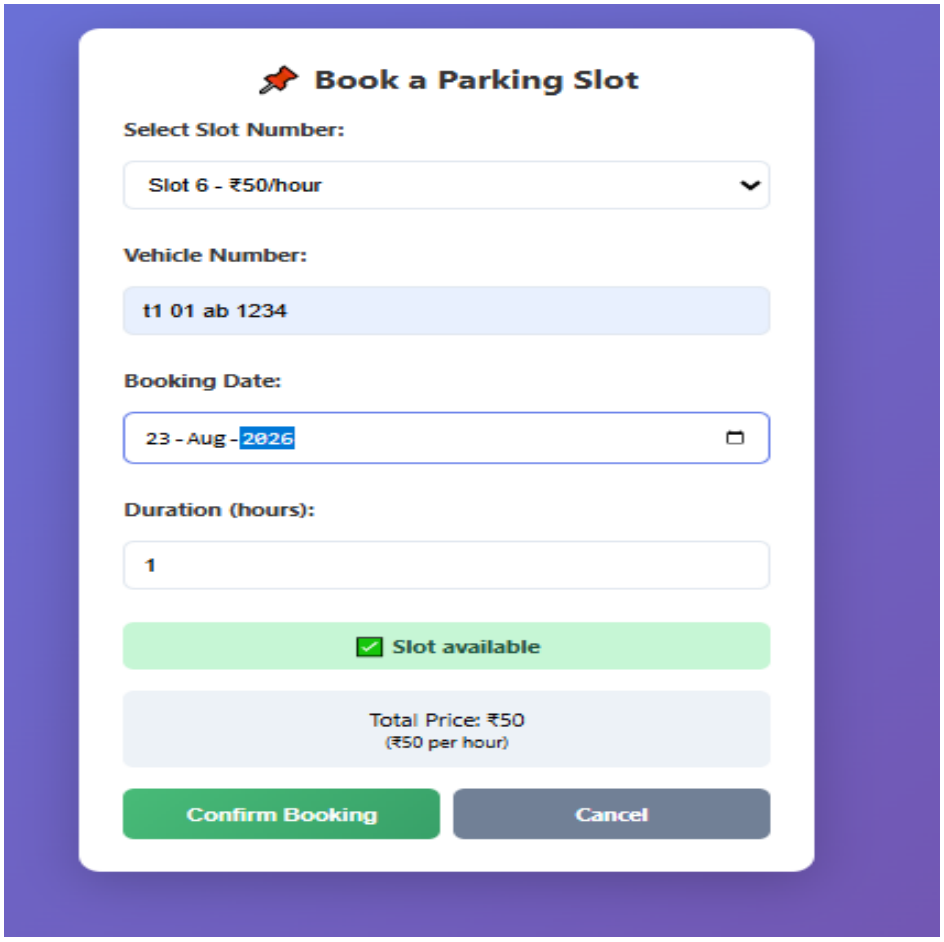
After login, users see available slots, booked slots, and their active bookings. Available slots are shown in green, booked slots in red. Users can click on any available slot to book



5.2.3 Slot Booking Module

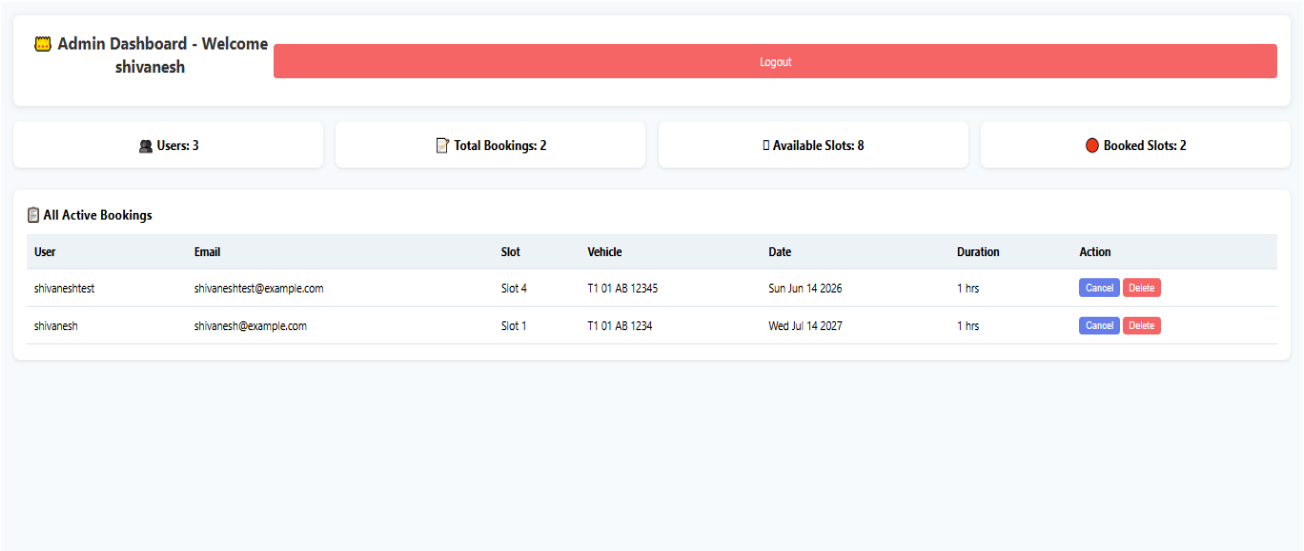
Users select slot number, enter vehicle number, choose date, and duration. The system checks availability for the selected date before confirming. A unique booking record is created in the database





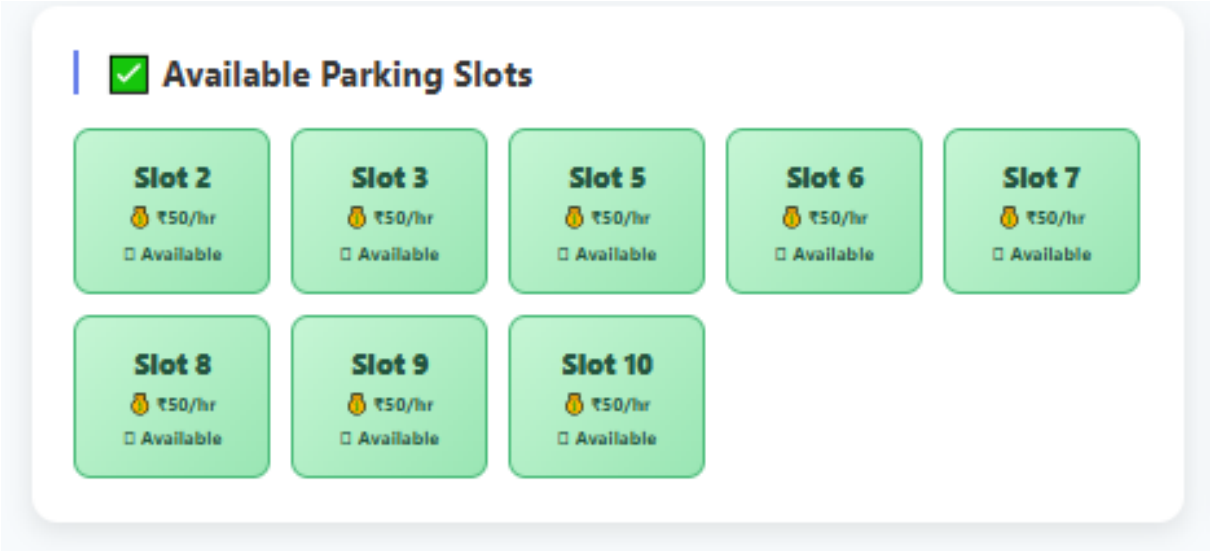
5.2.4 Admin Dashboard Module

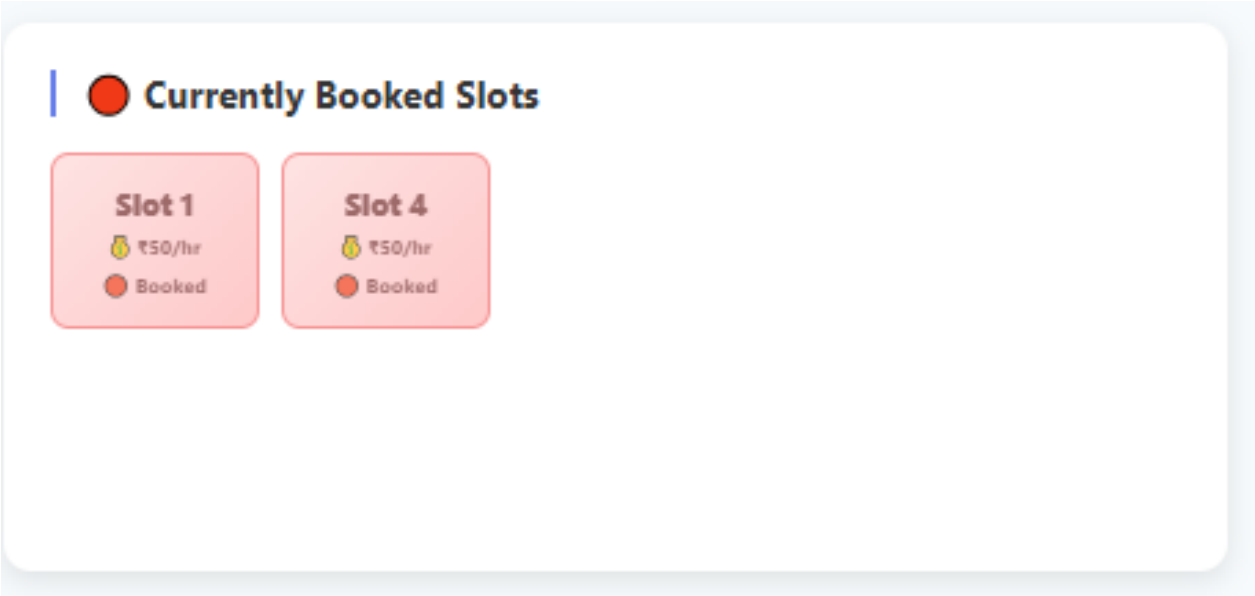
Administrators can view all active bookings from all users, including usernames, emails, slot numbers, vehicle numbers, dates, and durations. Admins can cancel or delete any booking.



5.2.5 Slot Availability Tracking

The system maintains 10 parking slots. When a booking is made, the slot is marked as booked for that date. Cancelled bookings free up the slot. Real-time availability is displayed on both user and admin dashboards.





5.3 Key Code Snippets

User Schema (models/User.js):

javascript

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' }
});
```

Booking Schema (models/Booking.js):

javascript

```
const bookingSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  slotNumber: { type: Number, required: true },
  vehicleNumber: { type: String, required: true },
  bookingDate: { type: Date, required: true },
  duration: { type: Number, required: true },
  status: { type: String, default: 'active' }
});
```

Booking Route (routes/user.js):

javascript

```
router.post('/book', isAuthenticated, isUser, async (req, res) => {
  const { slotNumber, vehicleNumber, bookingDate, duration } = req.body;
  const booking = new Booking({
    userId: req.session.userId,
    slotNumber, vehicleNumber, bookingDate, duration
  });
  await booking.save();
  res.redirect('/user/dashboard?success=Booking confirmed');
});
```

5.3 Module Descriptions

5.3.1 User Authentication Module

Files: routes/auth.js, models/User.js

Features:

Registration with validation

Login with password hashing

Session management

Logout functionality

5.3.2 User Dashboard Module

Files: routes/user.js, views/userDashboard.ejs

Features:

Display available slots (green)

Display booked slots (red)

Show user's active bookings

Cancel booking option

Statistics cards

5.3.3 Slot Booking Module

Files: routes/user.js, views/bookSlot.ejs

Features:

Slot selection (1-10)

Vehicle number input

Date picker (future dates only)

Duration selection (1-24 hours)

Real-time availability check

Price calculation

5.3.4 Admin Module

Files: routes/admin.js, views/adminDashboard.ejs

Features:

View all active bookings

View all registered users

Cancel any booking

Delete any booking

Statistics dashboard

Slot occupancy view

5.3.5 Slot Management Module

Files: models/Slot.js

Features:

Initialize 10 slots

Track availability

Price per hour: ₹50

5.5 Algorithm/Pseudocode

Booking Algorithm:

Algorithm: Book Parking Slot

Input: userId, slotNumber, vehicleNumber, bookingDate, duration

Output: Booking confirmation or error

```
1. Start
2. Validate input data
   IF any field is empty THEN
      Return "All fields required"
3. Check if bookingDate is not in past
   IF bookingDate < currentDate THEN
      Return "Cannot book past dates"
4. Check if slot is already booked for given date
   SELECT * FROM bookings
   WHERE slotNumber = input.slotNumber
   AND status = 'active'
   AND bookingDate = input.bookingDate
   IF record exists THEN
      Return "Slot already booked"
5. Calculate totalPrice = duration * 50
6. Create new booking record
7. Save to database
8. Return "Booking confirmed"
9. End
```

Availability Check Algorithm:

Algorithm: Check Slot Availability

Input: slotNumber, date

Output: Available (True/False)

```
1. Start
2. FOR each booking in bookings:
   IF booking.slotNumber == slotNumber
     AND booking.status == 'active'
     AND booking.bookingDate == date THEN
        Return False
3. Return True
4. End
```

CHAPTER 6: RESULTS AND DISCUSSION

6.1 System Output Screens

The system produces the following output screens:

Login Page: Users and admins enter credentials

Register Page: New user account creation

User Dashboard: Displays available slots (green), booked slots (red), and user's active bookings with cancel option

Booking Form: Form to select slot, enter vehicle number, date, and duration

Admin Dashboard: Displays all user bookings, statistics, and admin controls

6.2 Performance Evaluation

Parameter	Result
Page Load Time	< 2 seconds
Booking Confirmation	Instant
Slot Availability Update	Real-time
Authentication Response	< 1 second

6.3 System Testing

Test Case	Expected Result	Actual Result
Valid login	Redirect to dashboard	PASS
Invalid login	Show error message	PASS
Book available slot	Booking saved	PASS
Book already booked slot	Show error	PASS
Cancel booking	Status updated	PASS
Admin view all bookings	All records shown	PASS
Admin cancel user booking	Booking cancelled	PASS

6.4 Code Implementation

1. Server Setup (app.js):

javascript

```
require('dotenv').config();
const express = require('express');
const session = require('express-session');
const mongoose = require('mongoose');

const app = express();

// Middleware
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public'));
app.set('view engine', 'ejs');

// Session
app.use(session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: true
}));

// MongoDB Connection
mongoose.connect(process.env.MONGODB_URI)
    .then(() => console.log('MongoDB connected'))
    .catch(err => console.error(err));
```

```
// Routes
app.use('/', require('./routes/auth'));
app.use('/user', require('./routes/user'));
app.use('/admin', require('./routes/admin'));
app.listen(3000, () => console.log('Server running on port 3000'));
```

2. User Schema:

javascript

```
const userSchema = new mongoose.Schema({
    username: { type: String, required: true, unique: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    role: { type: String, default: 'user' }
});

userSchema.pre('save', async function(next) {
    if (!this.isModified('password')) return next();
    this.password = await bcrypt.hash(this.password, 10);
    next();
});
```

3. Booking Schema:

javascript

```
const bookingSchema = new mongoose.Schema({
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    slotNumber: { type: Number, required: true },
```

```
    vehicleNumber: { type: String, required: true },
    bookingDate: { type: Date, required: true },
    duration: { type: Number, default: 1 },
    status: { type: String, default: 'active' }
});
```

4. Booking Route:

javascript

```
router.post('/book', isAuthenticated, isUser, async (req, res) => {
    try {
        const { slotNumber, vehicleNumber, bookingDate, duration } = req.body;

        // Check if slot is already booked
        const existing = await Booking.findOne({
            slotNumber,
            status: 'active',
            bookingDate: { $gte: new Date(bookingDate) }
        });

        if (existing) {
            return res.render('bookSlot', { error: 'Slot already booked' });
        }

        const booking = new Booking({
            userId: req.session.userId,
            slotNumber,
            vehicleNumber,
            bookingDate,
            duration
```

```
        });

        await booking.save();
        res.redirect('/user/dashboard?success=Booking confirmed');
    } catch (error) {
        res.render('bookSlot', { error: 'Booking failed' });
    }
});
```

6.5 Problem Statement

Scenario 1: A user drives to a mall, spends 20 minutes finding parking, only to find it full.

Scenario 2: A parking admin manually records bookings in a register, leading to double bookings and disputes.

Scenario 3: No historical data to analyze peak hours and optimize pricing.

6.6 Problem Identification

1. Lack of real-time availability information
2. No advance booking mechanism
3. Manual record-keeping errors
4. No central monitoring for admins
5. Difficulty in handling cancellations
6. No data for analytics

6.7 Objectives of the System

Primary Objectives:

To develop a web-based parking booking platform

To provide secure authentication for users and admins

To display real-time slot availability

To enable advance slot booking

Secondary Objectives:

To allow users to cancel bookings

To provide admin dashboard for all bookings

To maintain centralized database

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Online Parking Management System was successfully designed and implemented as a web-based application to automate parking slot booking and management. The system allows users to view real-time slot availability, book slots for specific dates and durations, and cancel their bookings. Administrators can monitor all active bookings across all users and perform cancellations or deletions as needed.

The system uses Node.js for backend processing, MongoDB for centralized data storage, and EJS with HTML/CSS for a responsive user interface. Role-based authentication ensures secure access for both users and administrators.

By replacing manual parking management methods with a digital platform, the system reduces manual effort, improves slot utilization, enhances user convenience, and provides transparent record-keeping.

7.2 Future Enhancements

Integration of online payment gateway (Razorpay, Stripe)

SMS/Email notifications for booking confirmation and reminders

QR code generation for entry/exit validation

Real-time slot sensors using IoT

Mobile application for Android and iOS

Analytics dashboard with occupancy trends and revenue reports

Time-based dynamic pricing (peak hours vs non-peak hours)

7.3. Challenges Faced

Learning Node.js and MongoDB

Session management

Real-time availability logic

UI responsiveness

7.4 Future Enhancements

Payment Integration: Razorpay/Stripe API

Email/SMS Notifications: Using Nodemailer, Twilio

QR Code Generation: For entry/exit

Mobile App: React Native/Flutter

IoT Integration: Real-time sensors

Analytics Dashboard: Charts, peak hours

Dynamic Pricing: Peak vs non-peak rates

Waitlist Feature: Notify when slot available

Multiple Parking Locations: Support for multiple sites

Monthly Subscription Plans: For regular users

APPENDICES

Appendix A: Complete Source Code Structure

```
parking-management-system/
├── models/
│   ├── User.js
│   ├── Booking.js
│   └── Slot.js
├── routes/
│   ├── auth.js
│   ├── user.js
│   └── admin.js
├── middleware/
│   └── auth.js
├── views/
│   ├── login.ejs
│   ├── register.ejs
│   ├── userDashboard.ejs
│   ├── adminDashboard.ejs
│   └── bookSlot.ejs
├── public/
│   └── style.css
├── .env
├── app.js
└── package.json
```

Appendix B: Environment Variables

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/parking_db
SESSION_SECRET=your_secret_key
```

Appendix C: Installation Commands

```
# Install Node.js dependencies
npm install express mongoose bcryptjs express-session ejs dotenv
# Run MongoDB
mongod
# Start application
node app.js
```